

LGP-21 MVP Document

I. Introduction

This document outlines the technical and functional architecture of FESTIVO, a platform developed by team LGP-21.

II. Requirements developed

The FESTIVO platform has been developed with the following user stories in mind, targeting the Attendees, Organizers & Venues and Creative Professionals user groups.

Attendee User Stories

1. *As an attendee, I want to filter events by location, date, and category so that I can easily find relevant events near me.*
2. *As an attendee, I want to see which friends are attending an event so that I can decide whether I want to join.*
3. *As an attendee, I want to view complete event pages with descriptions, media, and reviews so that I can make informed decisions.*
4. *As an attendee, I want to post photos and videos with mandatory event tagging so that my content contributes to event galleries.*
5. *As an attendee, I want to follow users and events so that I can see their updates in my social feed.*
6. *As an attendee, I want to use a simple "Buy Ticket" button so that I can easily access external ticket providers.*

Organizer User Stories

1. *As an organizer, I want to create and edit event pages so that I can promote my events effectively.*
2. *As an organizer, I want to manage a tiered guest list (VIP, DJ, workers, guests) so that I can replace printed lists during check-ins.*
3. *As an organizer, I want to view RSVP statistics so that I can estimate attendee turnout.*
4. *As an organizer, I want to assign workers and performers to my events so that attendees can see who is participating.*
5. *As an organizer, I want to post job opportunities so that I can hire workers for upcoming events.*

Worker / Professional User Stories

1. *As a worker, I want to create a professional profile with a portfolio so that organizers can find and hire me.*
2. *As a worker, I want to browse and apply for job offers so that I can find relevant opportunities.*
3. *As a worker, I want to receive ratings and reviews so that I can improve my visibility and credibility.*
4. *As a worker, I want event assignments to appear on my profile so that my experience is publicly visible.*

III. Architecture

FESTIVO is a *full-stack* website, comprising a frontend, a backend server and a database. It also makes use of *Docker*, a containerization software, for quick and easy setup for testing and production.

Code organization

In terms of source code structure, the backend is organized around its layers. It contains a folder for each of `services`, `routes`, `middleware`, as well as a database handler `db.ts` and a general server file `server.ts`.

The frontend source code molds its structure around the `pages`, `components`, `shared utils` and a registration context for client authorization.

Core components and responsibilities

The backend **entrypoint** initializes the `Node.js` `Express` application, imports useful `JSON` parsing and `CORS` utilities, mounts routes and does a few other initial tasks.

In terms of **routing**, API endpoints are organized by subject. Overall, we have:

auth - (`/auth`, `/login`, ...), encapsulates authorization and authentication routes

events - (`/events`, `/events/recommended/:userId`, ...), related to event navigation, discovery and requests

social feed, interactions, friends - (`/feed`, `/publications/`, ...), your usual social-media-like functionality

chat - (`/chat/initiate`, `/chat/messages/:chatId`, ...), for messaging related functionality

work opportunities - (`/work/opportunities`, ...), job-search routes

The **service layer** handles business logic and `SQL` queries (`auth`, `events`, `social`, `work`, `users`, ...). This is the heart of the backend and is in charge of performing all operations to complete and serve user requests, including interacting with the `SQL` database.

The **frontend shell and routing** leverages the `React Router` technology to map all pages to the `URI` routes. We use consistent page layout with the aid of `React`'s modular approach to frontend development with components.

Frontend state is useful for keeping authorization tokens and other data, such as community interactions.

Frontend / backend separation

There is a clear 3-tier separation among frontend and backend, with the following ports being utilized:

Frontend on port 5173

Backend API on port 3000

PostgreSQL on port 5432

IV. Technologies used / Tech stack

Frontend:

- `React`
- `TypeScript`

- Vite
- React Router
- Tailwind plugin
- Leaflet

Backend:

- Node.js
- TypeScript
- Express
- pg
- bcrypt
- jsonwebtoken
- dotenv
- cors

Database:

- PostgreSQL schema
- triggers
- seed/populate SQL

Deployment / dev runtime:

- Dockerfiles/docker-compose